

Cloud Agents in the UI

Module 6 · Day 2 (Hands-On + Demonstration)

Cursor Training Program · ~90 min

Module Overview

Aspect		Details
Duration		~90 minutes
Format		Hands-on exercise + demonstration
Prerequisites		Cursor account, GitHub repository access, Modules 1–5 completed
Module Goal		Master Cloud Agents UI for remote execution, artifact collection, and messaging integrations

Learning Objectives

By the end of this module, participants will be able to:

- Launch and monitor Cloud Agents from the Cursor UI
- Collect and download artifacts from completed cloud runs
- Trigger Cloud Agents from messaging platforms (Slack, Discord)
- Manage cloud agent history and settings

Agenda

Lesson	Topic	Time
6.1	Launching a Cloud Agent	25 min
6.2	Cloud Agent Artifacts	23 min
6.3	Cloud Agents from Messaging Platforms	20 min

Lesson 6.1

Launching a Cloud Agent

Concept · 10 min · Exercise · 15 min

Cloud Agents vs. Local Agent

Aspect	Local Agent	Cloud Agent
Runs on	Your machine	Cursor's infrastructure
Persistence	Ends when you quit	Continues indefinitely
Access	Local only	Web, mobile, API
Terminal access	Your terminal	Simulated/scripted
File access	Local files	GitHub repos only
Best for	Interactive work	Batch, scheduled, hands-off

When to Use Cloud Agents

Good for:

- Long-running tasks (>10 min) • Scheduled jobs
- Tasks while offline • Parallel execution
- Team-accessible results (share agent URL)

Bad for:

- Interactive debugging • Local-only files
- Security-sensitive code • Quick questions

Accessing Cloud Agents UI

Method	Steps
From Cursor Editor	View → Cloud Agents (or cloud icon in sidebar)
From Web	https://cursor.com/agents
From Mobile	cursor.com/agents (responsive web)

Cloud Agent Dashboard

Active (2)

 security-audit-2024	running • 12 min elapsed
 doc-generator	running • 3 min elapsed

Completed (4)

 pr-review-42	FINISHED • 2 artifacts
 test-suite	FINISHED • 1 artifact

Failed (1)

 deploy-staging	ERROR • Auth token expired
--	----------------------------

Exercise 6.1 – Steps 1-2

Step 1: Navigate to Cloud Agents

```
# Cursor Editor: cloud icon or View → Cloud Agents  
open https://cursor.com/agents
```

Step 2: Click "+ New" and fill out:

```
Repository: https://github.com/YOUR_ORG/YOUR_REPO  
Branch: main  
Prompt: Read README and main source files. Summarize:  
  - What this project does  
  - Key dependencies · How to run locally · Common issues  
Model: claude-4.6-sonnet  
Auto-create PR: ☐
```

Exercise 6.1 – Steps 3-4

Step 3: Monitor live log in real time:

```
[10:45:01] Agent starting...
[10:45:02] Cloning repository...
[10:45:15] Repository cloned
[10:45:16] Reading README.md
[10:45:40] Generating summary...
```

Step 4: Configure settings (gear icon):

Setting	Purpose
Default Model	Preferred model for new agents
Auto-create PR	Create PRs on completion
Notification Email	Completion notifications
Webhook URL	POST completion events
Max Run Time	5 min - 24 hrs

Exercise 6.1 – Steps 5–6

Step 5: Launch with PR creation:

```
Prompt: Add CONTRIBUTING.md with dev setup, tests, PR process, code style  
Auto-create PR: ☒ Yes  
Branch prefix: docs/contributing
```

Step 6: Share agent URL with team:

```
https://cursor.com/agents/agt\_abc123def456
```

Success Criteria: Launched agent • monitored logs • configured settings • shared URL

Lesson 6.2

Cloud Agent Artifacts

Concept · 8 min · Exercise · 15 min

Types of Artifacts

Artifact Type	Examples
Log files	agent.log, debug.log
Code files	*.py, *.js, *.html
Documents	*.md, *.txt, *.json
Images	*.png, *.jpg, *.svg
Archives	*.zip, *.tar.gz
Test results	junit.xml, coverage.json

“ Files produced by the agent that you can download or view in the UI.”

Artifact Storage

- Stored for **30 days**
- Multiple artifacts per agent
- Download URLs expire after **15 minutes**
- Max **100MB** per file • **1GB** total per agent

Exercise 6.2 – Steps 1-2

Step 1: Launch agent that generates artifacts:

Generate:

1. api_documentation.md – OpenAPI-style docs for all endpoints
2. test_report.json – test suite summary
3. screenshot.png – main UI screenshot (if applicable)
4. dependencies.txt – all packages and versions

Place all in artifacts/ directory.

Step 2: After completion, view artifact list in UI with Download buttons and **Download All (zip)**

Exercise 6.2 – Steps 3–5

Step 3: Download individual artifacts

Step 4: Download all as zip

Step 5: Preview in browser:

- Markdown → rendered HTML
- Images → inline preview
- JSON → formatted tree view

Exercise 6.2 – API Access

```
# List artifacts
curl -s -u "$CURSOR_USER_API_KEY:" \
  "https://api.cursor.com/v1/agents/$AGENT_ID/artifacts" | jq '.'

# Download specific artifact
DOWNLOAD_URL=$(curl -s -u "$CURSOR_USER_API_KEY:" \
  ".../artifacts/download?path=artifacts/report.md" | jq -r '.url')
curl -L -o report.md "$DOWNLOAD_URL"
```

Create `bin/process-artifacts.sh` to batch-download all artifacts for an agent ID.

Exercise 6.2 – CI/CD Integration

```
# GitHub Actions – download test results from completed agent  
- name: Download Cloud Agent artifacts  
  run: |  
    curl -s -u "${{ secrets.CURSОР_API_KEY }}" \  
      ".../artifacts/download?path=test_results.xml" > test_results.xml
```

Success Criteria: Generated artifacts · downloaded single + zip · accessed via API

Lesson 6.3

Cloud Agents from Messaging Platforms

Concept · 10 min · Demonstration

Supported Integrations

Platform	Capabilities	Setup
Slack	Command triggering, notifications	Medium (Slack app)
Discord	Command triggering, webhook responses	Medium (Bot token)
Teams	Webhook integration	Medium (Webhook)
Generic Webhook	POST-triggered agents	Low (any platform)

Messaging Integration Architecture

```
Slack/Discord/Teams
| "/cursor review PR #42"
↓
Webhook Receiver (your server or Cursor hosted)
| POST /v1/agents with webhookUrl
↓
Cursor Cloud Agent
| POST to webhookUrl when complete
↓
Webhook Receiver → Post response to Slack
```

Demo: Slack Integration

Step 1: Create Slack App at api.slack.com

Step 2: Configure slash command:

```
Command: /cursor  
Request URL: https://your-server.com/webhook/slack-cursor  
Usage Hint: [prompt or command]
```

Step 3: Deploy webhook receiver (Flask/Python) that:

- Parses Slack command → launches Cloud Agent via API
- Acknowledges immediately with agent URL
- Posts completion summary when webhook fires

Demo: Slack Usage

In Slack:

```
/cursor Review the recent commits and summarize what changed
```

Response:

```
🤖 Launching Cloud Agent `agt_abc123`  
Watch progress: https://cursor.com/agents/agt\_abc123  
  
✅ Cloud Agent Complete!  
Summary: 3 commits – fixed login bug, added tests, updated README.  
PR: https://github.com/your-org/your-repo/pull/43
```

Demo: Discord Integration

```
@bot.command(name='cursor')
async def cursor_command(ctx, *, prompt):
    response = requests.post("https://api.cursor.com/v1/agents", ...)
    await ctx.send(f"✅ Agent launched: https://cursor.com/agents/{agent_id}")
```

Usage: `!cursor Add error handling to all API endpoints`

Generic Webhook & Notifications

Any HTTP POST can trigger agents:

```
curl -X POST https://your-server.com/trigger-agent \  
-H "Content-Type: application/json" \  
-d '{"prompt": "Run the weekly security scan", "repo": "..."}'
```

Use cases: GitHub webhook on PR · Cron jobs · CI/CD post-deploy · Internal dashboard

Status notifications: configure `notifyOnStart`, `notifyOnComplete`, `notifyOnError`

Success Criteria: Understood architecture · saw Slack/Discord demos · webhook triggering

Module Summary

Lesson	Topic	Key Skill
6.1	Launching Cloud Agents	Remote execution
6.2	Cloud Agent Artifacts	Output collection
6.3	Messaging Integrations	Chat-triggered agents

Quick Reference Card

UI ACCESS:

Cursor Editor: View → Cloud Agents
Web: <https://cursor.com/agents>

LAUNCH: + New → repo/branch → prompt → model → Launch Agent

ARTIFACTS: View in detail page · Download single/zip · API (30-day storage)

MESSAGING:

/cursor [prompt] in Slack
!cursor [prompt] in Discord
Webhook POST to trigger endpoint

Up Next: Module 7

Model Configuration

“ Now that you've mastered Cloud Agents in the UI, **Module 7** covers selecting the right models, managing API keys, and optimizing cost vs. quality. ”

End of Module 6